



The Hopfield Network - A Simple Python Example

Written by Mike James
Monday, 14 October 2024

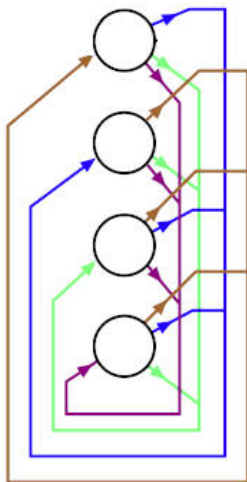
Article Index

The Hopfield Network - A Simple Python Example (/programming/102-artificial-intelligence/5097-the-hopfield-network-a-simple-python-example.html?utm_source=perple
Exploring Hopfield Networks (/programming/102-artificial-intelligence/5097-the-hopfield-network-a-simple-python-example.html?utm_source=perplexity&start=1)
Python Program (/programming/102-artificial-intelligence/5097-the-hopfield-network-a-simple-python-example.html?utm_source=perplexity&start=2)

Page 2 of 3

The Hopfield Network

To understand how the program works you have to know exactly how the Hopfield neural network functions. A Hopfield network is a single layer of artificial neurons with every neuron connected to every other neuron, that is it is a completely connected network.



Each neuron can be in one of two states - +1 or -1. You can think of +1 as the neuron firing and -1 as it being relaxed. Although all of the neurons are connected not all of the connections have the same strengths and it is by adjusting the values of the strengths that the network learns.

If the network consists of m neurons then the state of the entire network can be recorded in a single array s with $s[i]$ holding the state (+1 or -1) of the i th neuron.

In the same way the strengths of the connections between all of the neurons can be stored in a two-dimensional array w with $w[i][j]$ holding the strength of the connection between neuron i and neuron j .

The rule for whether or not a neuron fires depends on the total of the inputs received. The total input to neuron i is simply:

$$a_i = \sum_j s[j] w[i][j]$$

i.e. the state of neuron j times the strength of the connection between neuron i and j summed over all neurons. Notice that the connection $w[i][i]$ is zero the connections are symmetric $w[i][j] = w[j][i]$.

The update rule for the network is that the new state of the neuron is

$$s_i = 1 \text{ if } a_i > 0 \\ = -1 \text{ if } a_i < 0$$

The way that the Hopfield network is used to recall something is that an input pattern I is used to set the initial state of the network, that is I is stored in s and then the network is allowed to evolve. The update rule is repeatedly applied to all of the neurons until the values settle down and there is no change after an update.

It is a characteristic of a Hopfield network that the network will always evolve towards a fixed state. The final state is taken as the output of the network i.e. what it recalls when the input (initial state) is I .

All that is left is to describe how to teach the network a pattern. This is done by adjusting the connection matrix by adding an increment to each term $w[i][j]$. The weight increment for the i,j th weight is

$$\text{inc} = s[i]s[j]$$

In other words the connection between neurons i and j are increased or decreases in proportion to the product of the i and j th elements of the pattern that the network is trying to learn.

It is not difficult to see that this rule produces a symmetric connection matrix. We also have to remember to enforce the condition that $w[i][j]=0$ to ensure that there is no connection between a neuron and itself!

This learning rule is generally referred to as the 'Hebbian learning rule'. It is not the only one possible but if you use it you can be assured that the network always settles down either to a fixed state or one that oscillates between two fixed states. If you are interested in experimenting with neural networks then you might like to try alternative rules.

You can get rid of the irritation of oscillating states by performing the update on the network asynchronously.

There are two ways of computing the new state of the network. You can take the strictly mathematical approach as contained in the equations given above and work out the new states for all the neurons based on all of the old values. If the network was realised in hardware this would be equivalent to making all the neurons change their state at the same moment - synchronously. The alternative is to allow only one neuron to change its state at a time - asynchronously. This means that at any given moment some of the neurons will have new states and some old and so the input to any given neuron will be a mixture of both. Asynchronous updates are to be preferred because they are easier to program because they require no additional storage and they can also be shown to avoid oscillation between final states.

Hopfield as Matrices

If you know your linear algebra the simplest way to appreciate the Hopfield algorithm is as a pair of simple matrix equations. If the state of the neurons is represented as a vector s , a vector with elements 1 or -1 and the weight matrix is W then the learning equation is:

$$W' = W + s \otimes s$$

where the operator is the outer product of vectors and s represents the state being remembered. You can also write this as

$$W' = W + ss^T$$

The update is performed once.

The recall equation is:

$$s' = \text{sign}(Ws)$$

where sign returns 1 or -1 depending on the sign of the result. This equation is iterated until it converges to a final value of s .

You can easily interpret this as a simple eigenvector computation. If we only store a single pattern the analysis is simple.

If x is the state you want to store then $W=xx^T$ has eigenvector x and eigenvalue n^2 as

$$Wx=xx^Tx= xn^2$$

and x^Tx is n^2 .

Is also well known that iterating Wz converges to the eigenvector corresponding to the largest eigenvalue which is in this case x . Thus the memory of the Hopfield net is a simply based on constructing a matrix with an eigenvector that corresponds to the pattern we are trying to remember.

If you store multiple patterns in the network then as long as the patterns are orthogonal (uncorrelated) then each can be stored and recalled without interference. For example for two patterns x and y we have:

$$W=xx^T+yy^T$$

and hence

$$Wx=xx^Tx+yy^Tx = n^2x$$

as y^Tx is zero.

So as long as x and y are uncorrelated they are both exact eigenvectors of W with eigenvalues n^2 . If you iterate W with an starting vector z then this converges to the closest eigenvector.

If the stored vectors are not uncorrelated the process still works as long as they are not too correlated:

$$Wx=xx^Tx+yy^Tx = n^2x + cy = n^2(x + c/n^2 y)$$

where c is $y^T x$ i.e. the correlation. It is easy to see that x will still be an eigenvector as long as $\text{abs}(c/n^2 y) < 1$.

Hopfield as Physics

How does all this relate to physics? The simple answer is that the neuron array is a spin glass. Each neuron can be thought of as a magnet with north up for a 1 and down for a -1. The weight matrix models the interaction between each of the "magnets" and the recall iteration changes the direction of the magnets to minimize the energy of the system. A spin glass moving towards lower energy is a cooling process and this is where the physics comes in.

Storage Capacity

You can see that when a Hopfield network learns a pattern the information about that pattern is distributed across the network as the strengths of the connection. This is very much like the way that we think the brain stores information. A difficult question to answer is how many patterns can be stored in a single network?

The answer depends on how dissimilar the patterns are.

In this case the appropriate measure of similarity between two patterns A and B is the sum of all terms like $A[i]*B[i]$. If you know a little statistics you will recognise this expression as being proportional to the correlation between the patterns A and B . It has been estimated that the storage capacity of the Hopfield network is roughly $.145*m$. In the case of our 64 neuron network this gives an upper limit to the storage of 9 uncorrelated patterns. Of course in practice the patterns would be correlated and in this case the actual storage capacity. This is relatively inefficient in that it can be proved that the optimal storage capacity for a network of M neuron is 2^M . In other words the Hopfield network is only 7% efficient. The key to improving its performance is to tinker with the learning rule.

<< [Prev \(/programming/102-artificial-intelligence/5097-the-hopfield-network-a-simple-python-example.html?utm_source=perplexity\)](/programming/102-artificial-intelligence/5097-the-hopfield-network-a-simple-python-example.html?utm_source=perplexity) - Next >> [\(/programming/102-artificial-intelligence/5097-the-hopfield-network-a-simple-python-example.html?utm_source=perplexity&start=2\)](/programming/102-artificial-intelligence/5097-the-hopfield-network-a-simple-python-example.html?utm_source=perplexity&start=2)

Last Updated (Saturday, 19 October 2024)